

Spark's Basic Architecture

Introduction

Apache Spark is a powerful distributed computing framework designed for fast processing of large-scale data. It follows a **master-worker architecture**, where a central node (Driver) coordinates tasks across multiple worker nodes.

Core Components of Spark Architecture

Driver Node

The **Driver Node** acts as the central controller of a Spark application.

Responsibilities:

- Executes the main program
- Creates execution plans (DAG – Directed Acyclic Graph)
- Divides tasks into smaller units
- Sends tasks to worker nodes
- Collects results

It is considered the **brain of the Spark application**.

SparkSession

SparkSession is the entry point to interact with Spark.

Features:

- Used to create DataFrames and Datasets
- Combines functionalities of SQLContext, HiveContext, and SparkContext
- Provides a unified interface for data processing

Example:

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder \
    .appName("MyApp") \
    .getOrCreate()
```

SparkContext

SparkContext is the core engine that connects the application to the cluster.

Functions:

- Establishes communication with the cluster manager
- Allocates resources
- Distributes tasks to executors

It acts as a **bridge between the Driver and the cluster**.

Cluster Manager

The **Cluster Manager** is responsible for managing resources in the cluster.

Responsibilities:

- Allocates CPU and memory resources
- Manages multiple applications
- Schedules jobs across worker nodes

Common Types:

- Hadoop YARN
- Kubernetes
- Spark Standalone Cluster

Worker Nodes

Worker nodes are machines that perform the actual data processing.

Responsibilities:

- Execute tasks assigned by the Driver
- Store intermediate data in memory or disk
- Return results to the Driver

These nodes contain **Executors**, which run the tasks.

Working of Spark Architecture

The execution flow in Spark is as follows:

1. User writes code using SparkSession
2. Driver Node processes the code
3. SparkContext sends tasks to the Cluster Manager
4. Cluster Manager allocates resources
5. Worker Nodes execute the tasks
6. Results are sent back to the Driver

Client Mode vs Cluster Mode

Client Mode

In Client Mode, the Driver runs on the user's local machine.

Characteristics:

- Suitable for development and testing
- Easy debugging

- Requires stable connection

Limitation:

- If the client machine fails, the job fails

Cluster Mode

In Cluster Mode, the Driver runs inside the cluster.

Characteristics:

- Suitable for production environments
- More reliable and scalable
- No dependency on client machine after submission

Key Differences:

Feature	Client Mode	Cluster Mode
Driver Location	Local Machine	Cluster
Reliability	Less	High
Use Case	Testing	Production

Feature

Client Mode

Cluster Mode

Dependency

Client dependent

Independent